

CORGE

SPECIFICATION-DRIVEN AGENTIC DEVELOPMENT

Human-Led · Machine-Assisted Engineering

Product Presentation · 2026

TEAM

Ahmad Fathi

Ahmad Hani

Hala Muhammad

Muhammad Nabil

Radwa Muhammad

AGENDA

01 The Problem

02 The Answer: Corge

03 Technical Architecture

04 Layer & Module Deep Dive

05 Summary

06 Future Work & Roadmap

07 Lessons Learned

01 **THE PROBLEM**

Agentic Coding Has an Audience Problem

THE AGENTIC CODING LANDSCAPE

GENERAL AUDIENCES

- Vibe-coding culture
- Prompt-and-pray workflow
- Zero engineering discipline

BLACK-BOX SDLC

- Agent does everything
- Requirements to Code in one shot
- Human = spectator

OPINIONATED & OPAQUE

- Cannot inspect the plan
- Cannot steer mid-execution
- Cannot say “no, not like that”

- **Every week** a new tool claims to build your app from a single prompt.
- But *who* are these tools actually built **for**?

THE IRONY

LLMS ARE GREAT AT

- Syntactically correct code generation
- API reasoning & pattern matching
- Working across massive training data

LLMS ARE TERRIBLE AT

- Requirements engineering
- Architectural judgment
- Knowing when to stop and ask

*“They’re brilliant at generating code, but they can’t tell you **what** to build or **why**.”*

THE INSIGHT

“The bottleneck is not code generation.
It’s **specification** and **verification**.”

Which led us to ask:

*What if we stopped treating the LLM as an autonomous engineer and started treating it as a **disciplined tool** in a human-led engineering workflow?*

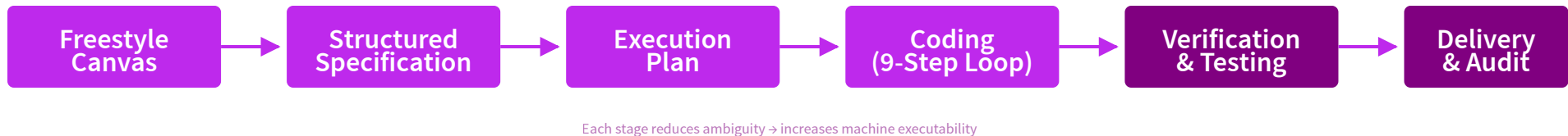
02

THE ANSWER: CORGE

Specification-Driven, Engineer-First

WHAT IS CORGE?

A **specification-driven, software-engineer-first** agentic development system. It enforces a controlled engineering workflow where ambiguity decreases at every stage.



THE THREE AGENTS

Agent	Phase	Role
Specification Agent	SPEC	Elicits requirements, runs Socratic Q&A, produces structured Specification with acceptance criteria
Planning Agent	PLAN	Generates TechnicalPlan (architecture) and ProceduralSteps (executable checklist)
Coding Agent	CODE	Executes the plan step-by-step — reads context, calls the LLM, requests human approval for every destructive action

Each agent has a strict responsibility. No overlap. No mission creep.

USER JOURNEY

1 Launch & Repo

- Point Corge at any repository or empty directory

2 Freestyle Canvas

- Rough ideas with `@node:` references validated in real time

3 Socratic Wizard

- Opt-in clarifying questions (capped at 3)

4 Split-Pane Editor

- Fill `[GAP: Topic]` templates, then approve

5 Technical Plan

- Architecture review — edit, approve, or reject (backward nav)

6 Procedural Steps

- Granular checklist with bracketed IDs

7 Step-by-Step Exec

- Context hydration → LLM → Approval with live `Ctrl+D` diffs

8 Verification & Done

- All criteria checked, tests pass, audit log delivered

BUSINESS VALUE

TRACEABILITY

- Every line of code traced to approved spec and plan step

SAFETY

- No write/edit/bash executes without human consent

PRIVACY

- No source code or context leaves the machine without user consent

QUALITY

- Spec-driven development prevents scope creep and ambiguity

AUDIT

- Full argumentation log + audit trail of every tool call and approval

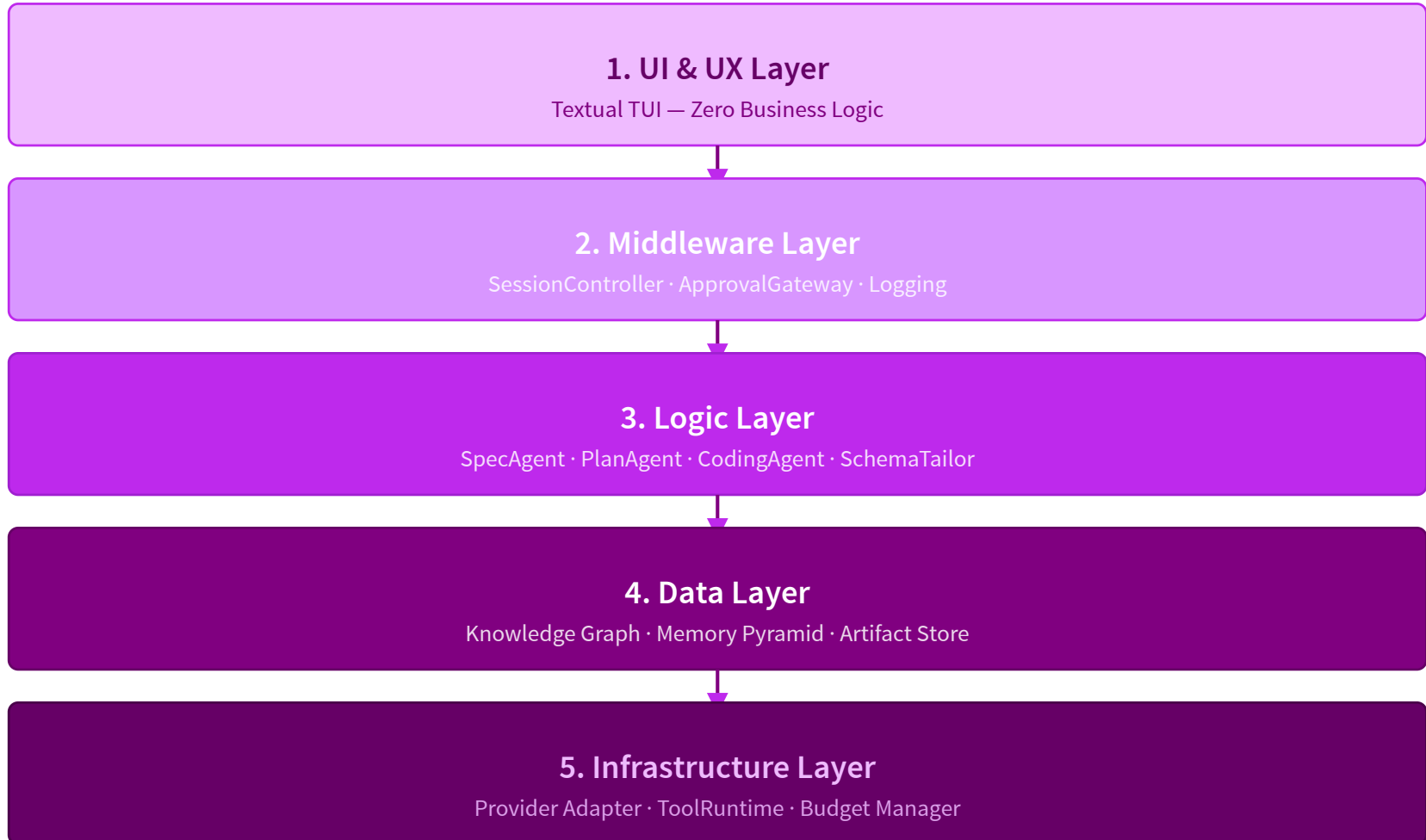
ACCOUNTABILITY

- The human is the engineer; the LLM is the tool

KEY DIFFERENTIATOR

- Not autonomous — collaborative. Human-led, machine-assisted.

ARCHITECTURE OVERVIEW



typing.Protocol interfaces between every layer

03

TECHNICAL ARCHITECTURE OVERVIEW

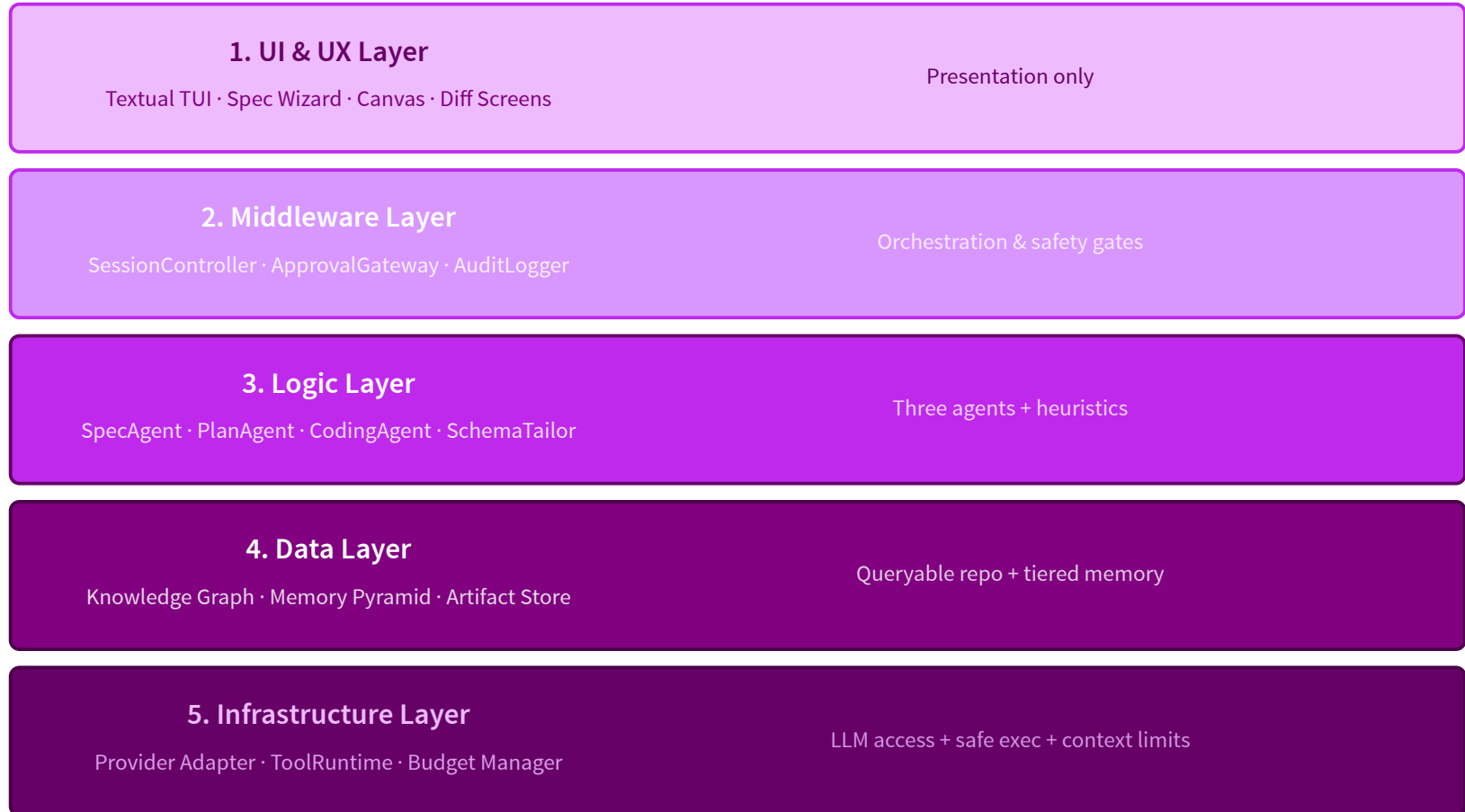
Stack, Layers, and Modularity

TECHNOLOGY STACK

Layer	Technology
Runtime	Python 3.11+, <code>uv</code> package manager
TUI Framework	Textual (rich terminal UI)
LLM Providers	DeepSeek, OpenAI, Ollama (OpenAI-compatible adapter)
Persistence	SQLite (WAL mode, connection pooling)
AST Parsing	Python stdlib <code>ast</code>
Static Analysis	<code>ruff</code> for linting, <code>mypy</code> for type checking
Testing	<code>pytest</code>

POC stack — deliberately minimal, no premature infrastructure.

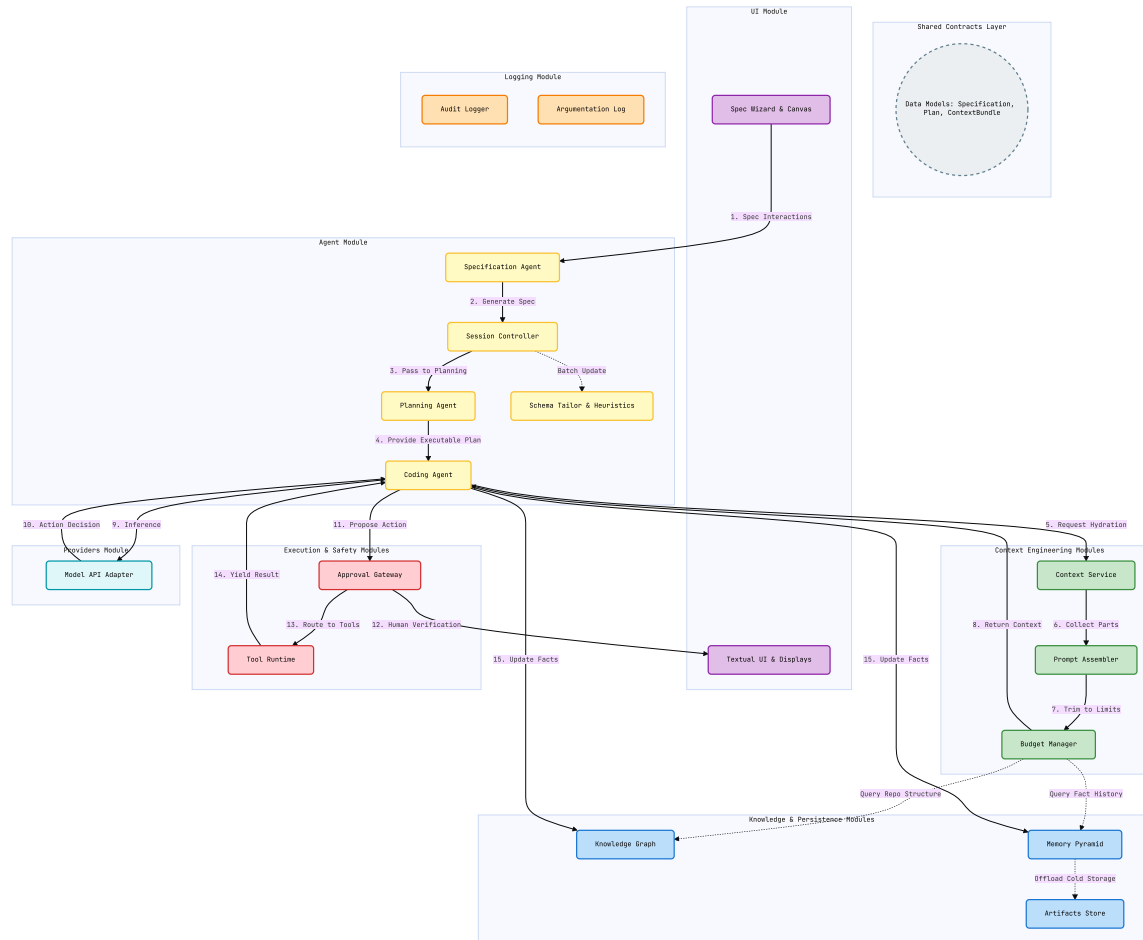
5-LAYER MODULAR MONOLITH



Protocol interfaces between every layer

SYSTEM MODULE MAP

Each numbered flow corresponds to the **15-step execution protocol** between modules.



MODULARITY SERVES THE BUSINESS

SWAPPABLE UI

- Replace Textual TUI with Web UI or IDE plugin — zero business logic changes
- UI module only depends on `UiPort` protocol; swap it out freely

TESTABLE IN ISOLATION

- Every module exposes a Protocol; mock the port, test the logic
- Integration tests verify contract compliance across boundaries

SWAPPABLE PROVIDERS

- Swap DeepSeek for Ollama or OpenAI with zero code changes
- Single `Provider` protocol adapter handles all model backends

NO TIGHT COUPLING

- UI has zero `import` from `agent` module
- Agents have zero `import` from `tools` or `providers`
- Compile-time safety via Protocol structural subtyping
- No dependencies on other modules, only on Protocols

CONTRACTS LAYER

- `typing.Protocol` defines every module boundary via port interfaces
- Enables the numbered execution flow from the module map

PARALLEL DEVELOPMENT

- UI team, agent team, and tooling team work independently behind protocol interfaces
- Contracts serve as the shared agreement — no coordination overhead per change

04 **LAYER & MODULE DEEP DIVE**

From the TUI to the Tool Runtime

LAYER 1: UI & UX

What: Pure presentation layer on Textual. *Zero business logic.*

KEY MODULES

- `CorgeApp` — Main application
- `DirectorySelectorApp` — Keyboard-first repo picker
- `CanvasScreen` — Sticky note canvas
- `InteractiveDiffScreen` — 4-context split pane
- `MessageScreen` / `ConfirmScreen`

DESIGN PRINCIPLES

- Keyboard-first: auto-focus on mount
- `Ctrl+D` = diff toggle, `Ctrl+A` = approve, `Escape` = reject
- `@work(thread=True)` keeps TUI responsive during LLM calls
- Swappable: UI is a view, not a controller

```
# CliUi implements UiPort protocol
class CliUi:
    def request_approval(self, request):
        return self._app.push_screen_wait(
            InteractiveDiffScreen(
                title="Approval Required", ...))
```

CANVASSCREEN & STICKY NOTE VALIDATION

`@node:<id>` references validate against the knowledge graph **as you type**.

KEY BEHAVIOR

- Validation is **debounced** — fires after user stops typing
- Valid reference: accepted silently
- Invalid reference: visible error indicator
- Unresolved stickies **block spec phase advancement**

Turns the canvas from a text dump into a **live knowledge graph-aware scratchpad**.

INTERACTIVEDIFFSCREEN — 4 CONTEXTS, ONE COMPONENT

Context	Left Pane	Right Pane	Read-Only?	Ctrl+D
Argumentation Diff	Raw canvas text	Concretized spec with [GAP:]	No (editable)	Diff vs canvas
Tech Plan Editor	Approved spec	Draft architecture plan	No (editable)	Diff vs previous
Steps Editor	Technical plan	Ordered step checklist	No (editable)	Diff vs original
Approval Gateway	Reason / code diff	Tool action payload	Yes	Toggle reason / diff

One parameterized component, infinite reuse.

CONFIRMSCREEN & TUI PATTERNS

A **15-line** confirmation dialog reused in three places:

1. SOCRATIC VALIDATION

“Semantic gaps detected. Run clarifying questions?”

Yes → wizard · No → skip

2. CLEAR BACKLOG

“Permanently erase
.agent/sticky_notes.json?”

Only Yes proceeds

3. TOOL RETRY

“Step failed. Retry after manual fixes?”

Yes re-runs · No suspends

```
# Every screen auto-focuses primary widget on mount
# Ctrl+A = approve, Escape = reject – consistent
```

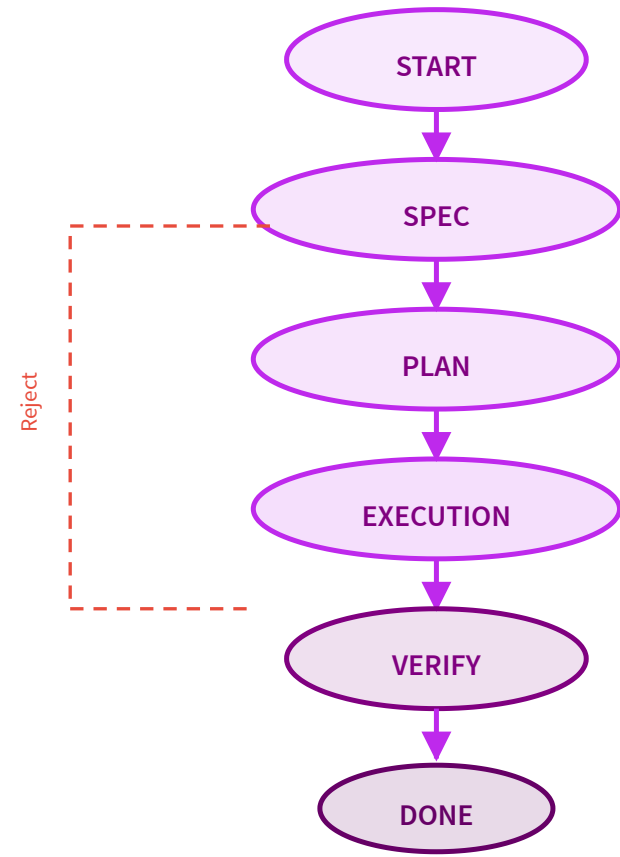
LAYER 2: MIDDLEWARE — SESSIONCONTROLLER

What: The master state machine driving all 3 phases across 12 lifecycle states.

TRANSITION TABLE (SIMPLIFIED)

- START → REPOSITORY_SELECTION → REPOSITORY_ANALYSIS
- SPEC_ENTRY → SPEC_VALIDATION → SPEC_APPROVAL
- PLAN_GENERATION → PLAN_REVIEW → PLAN_APPROVAL
- EXECUTION → VERIFICATION → COMPLETION_REVIEW → DONE

Backward navigation on rejection — guaranteed by centralized transition table.



State saved to .agent/ before each transition

APPROVALGATEWAY & AUDIT LOGGER

APPROVALGATEWAY

Single chokepoint for all code-modifying operations. **35 lines.**

```
class ApprovalGateway:
    def approve(self, request) → ApprovalDecision:
        if request.action == ToolAction.READ:
            return APPROVED
        decision = self._ui.request_approval(request)
        self._audit_logger.record_approval(request, decision)
        return decision
```

AUDIT LOGGER

Dual-stream logging:

- **Local:** `.agent/audit.jsonl` — tool-level detail
- **Global:** `~/.config/corge/global_audit.jsonl` — cross-project milestones

SAFETY BY CONSTRUCTION

- Read auto-approves
- Write/Edit/Bash blocks for human consent
- Live `Ctrl+D` code diffs before any action
- Every decision is audited

SELF-IMPROVING AGENTS

The system learns from every session. After each project, a **batch-phase heuristic updater** analyzes the argumentation log and adjusts the spec wizard's behavior for future sessions.

HOW IT WORKS

- Records every Socratic Q&A interaction
- Tracks which questions users override or skip
- Adjusts question priority for next session
- No manual tuning — self-calibrating

ABANDONMENT PROTECTION

- If a user quits mid-spec, the system penalizes its engagement prior
- Prevents over-asking in future sessions
- Single bad session can only degrade the model slightly

CONSERVATIVE DRIFT PROTECTION

- A single session can only shift behavior by a limited amount
- Overrides require multiple consistent sessions to shift behavior
- After 20 successful sessions with zero overrides, the wizard converges to a stable, minimal-intervention cadence

KEY TAKEAWAY

- The spec wizard gets **better over time** without human reconfiguration
- Self-improvement through usage data, not hardcoded rules

LAYER 3: LOGIC — THE THREE AGENTS

SPECIFICATION AGENT

- Canvas text → Structured Specification
- SchemaTailor: auto-detect framework
- Socratic Q&A (opt-in, capped)
- Inline [GAP: Topic] placeholders

PLANNING AGENT

- Spec → TechnicalPlan
- Architecture markdown
- ProceduralSteps with bracketed IDs
- Human approves design before implementation

CODING AGENT

- 9-step execution loop per step
- Observe → Refresh → Assemble → LLM → Approve → Execute → Verify → Update → Repeat
- Tool failure → Retry dialog → Scenario memory

WORKFLOW STATE CHAINING & CONTEXT ISOLATION

STEP-TO-STEP STATE MACHINE

After each execution step, the system captures what the agent proposed and how the human responded. This state is injected into the next step's prompt — the agent **learns from its corrections** mid-session.

```
# State captured between steps
step_state = {
  "proposal": "Add route /health in app.py",
  "correction": "Use /api/v1/health instead",
  "trajectory": "...previous steps..."
}
```

3-LAYER CONTEXT ISOLATION

Each phase gets only the context it needs. Instruction pollution is prevented by design.

Phase	Included	Omitted
Spec	Canvas, schema, Q&A	File lists, AST, code
Plan	Spec, repo tree, AST, config	Execution logs, tool outputs
Code	Spec, plan, graph, memory, artifacts	Canvas drafts, Q&A details

LAYER 4: DATA — KNOWLEDGE GRAPH

SQLite-backed AST parser building a `nodes` + `edges` graph of the repository.

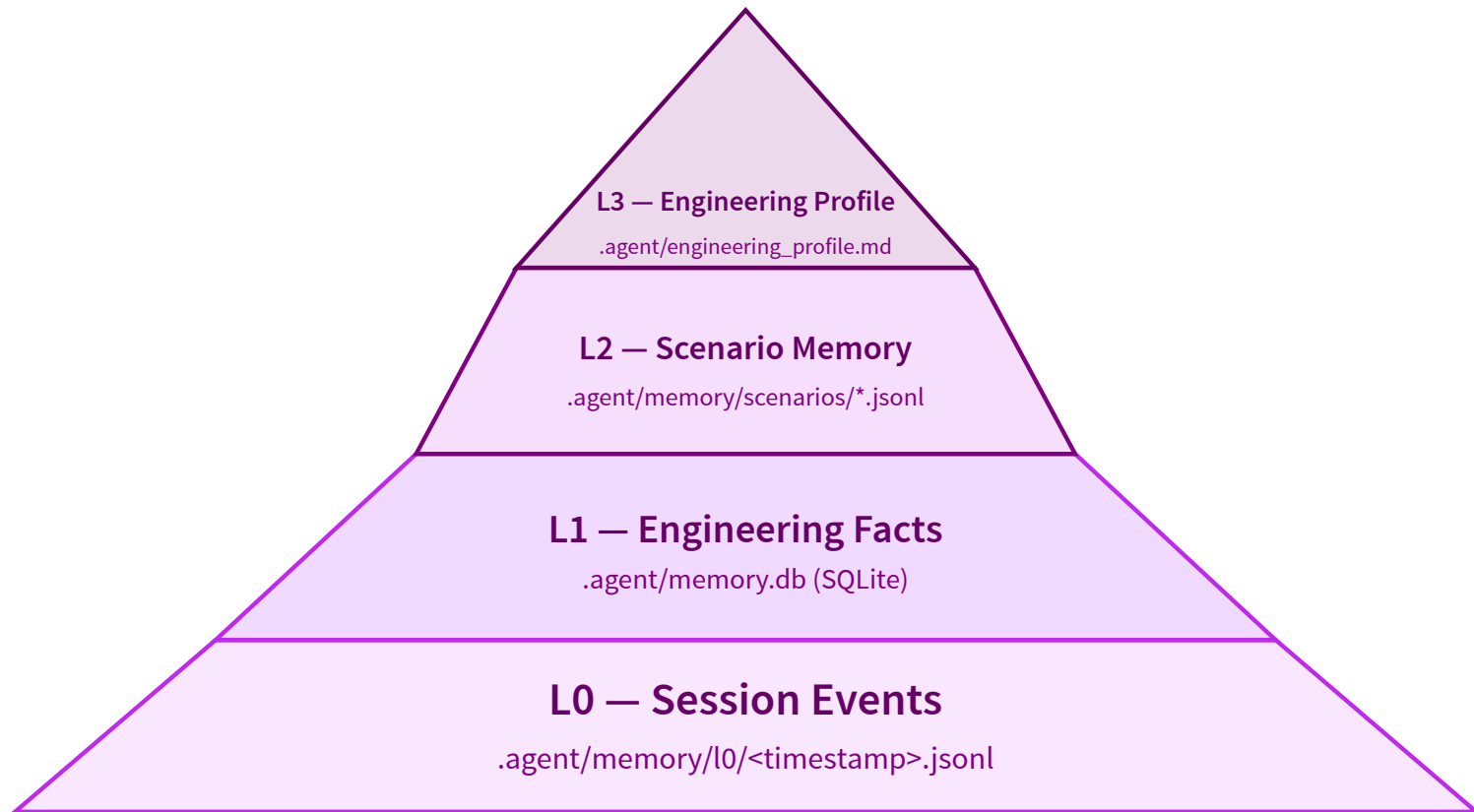
QUERY GRAMMAR

- `files` — List all files
- `classes:<path>` — Classes in a file
- `imports:<path>` — Dependencies
- `imported_by:<path>` — Reverse deps
- `node:<node_id>` — Specific node

The LLM cannot navigate a filesystem — the knowledge graph gives it a **queryable representation**.

```
CREATE TABLE nodes (  
  node_id TEXT PRIMARY KEY,  
  kind    TEXT NOT NULL,  
  path    TEXT NOT NULL,  
  name    TEXT NOT NULL  
);  
CREATE TABLE edges (  
  src TEXT NOT NULL,  
  rel TEXT NOT NULL,  
  dst TEXT NOT NULL,  
  PRIMARY KEY (src, rel, dst)  
);
```

MEMORY PYRAMID



4-tier pyramid: L0 raw logs → L3 confidence-filtered conventions

LAYER 5: INFRASTRUCTURE

PROVIDER ADAPTER

Single OpenAI-compatible adapter for supported provider families like:

- **DeepSeek** — `deepseek-*` model prefix
- **OpenAI** — Standard endpoint
- **Ollama server** — Local `API`
- **OpenRouter** — `sk-or-*` key prefix

TOOL RUNTIME

Four stateless primitives with safety validation:

- `read` — File reading
- `write` — File writing
- `edit` — Exact occurrence-count matching
- `bash` — Sandboxed, no `sudo`/traversal

BUDGET MANAGER

Unconditional transcript compaction:

- Duplicate removal
- Large-string clipping
- Context window ceiling enforcement
- LLM sees most relevant context

05 **SUMMARY**

What We've Built

13 ACHIEVEMENTS

✓ **Spec-driven workflow** — full backward nav on rejection

✓ **Interactive TUI** — keyboard-first, auto-focus, `Ctrl+D` diffs

✓ **Three-phase agents** — Spec, Plan, Code with strict boundaries

✓ **Approval Gateway** — read auto-approves; writes block for consent

✓ **Knowledge Graph** — SQLite-backed AST parser with query grammar

✓ **Memory Pyramid** — L0-L3 with confidence-filtered profiles

✓ **Provider Abstraction** — single adapter for 3 providers

✓ **Safety Validation** — no sudo, no system file redirects

✓ **Audit System** — dual-stream + argumentation log

✓ **Self-Improving Wizard** — spec wizard learns from usage over time

✓ **Session Persistence** — full save/resume across restarts, State machine controlled

✓ **Artifact Offloading** — `artifact://` URIs prevent context bloat

✓ **Modular Monolith** — 8 main modules via `~.Protocol`, no runtime deps

06

FUTURE WORK & ROADMAP

Deferred Until Real Usage Demands It

ROADMAP

Area	Current	Planned
Git Integration	No git awareness	Transactional rollbacks, incremental graph updates
IDE Plugins	Standalone TUI only	VS Code / JetBrains via JSON-RPC
GitHub Integration	Local TUI approvals	PR automation, remote approvals
Containerization	<code>uv run</code> on host	Docker multi-stage with sandboxed execution
Vector Search	SQLite <code>LIKE</code> scans	Semantic search via Qdrant/Chroma
Multi-Agent	Linear SessionController	Event-driven decentralized agents
Cloud Persistence	Local <code>.agent/</code>	PostgreSQL + S3 for team workspaces
Background Tasks	Synchronous TUI thread	Cron-scheduled reindexing, GC, learning

Every deferral names its ceiling and upgrade path. Built for POC exit criteria, not hypothetical scale.

07 **LESSONS LEARNED**

From Ideation to Architecture

7 LESSONS

1 Specs-first sounds slow; it's faster.

The Socratic wizard catches ambiguities before a line of code is written.

2 Protocol-driven modularity works.

`typing.Protocol` enabled parallel development across teams.

3 Hardest part is context engineering.

Getting the right info into the prompt, within token limits.

4 Human-in-the-loop is a quality feature.

Every rejection teaches the system. The heuristic updater captures this.

5 Keyboard-first TUI is worth the effort.

Engineers live in the terminal. Corge feels native.

6 YAGNI

Every deferred feature has a named ceiling and upgrade path.

7 Modularity is key

Modular software is not just for POCs.

THANK YOU

Questions?

Corge — Specification-Driven Agentic Development
Human-Led · Machine-Assisted